

2025年CSP-J模拟3--爬虫编程

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 被誉为“中国首款 3A 游戏”的《黑神话：悟空》自 2024 年发售以来广受好评，成为国产游戏历史上的现象级作品。

发售仅三天，销量即突破 1000 万套，多次荣登年度游戏榜单，并斩获多个最佳视觉设计与剧情奖项。

为呈现顶级画质与打击感，该游戏在高画质模式下，每秒渲染 60 帧画面，每帧分辨率为 2560×1440 ，每个像素占用 4 字节（*RGBA* 通道）。

假设以**未压缩原始图像格式**进行保存，不考虑音效及其他系统数据，若玩家连续游玩 5 分钟，显卡理论上共处理的图像数据大小为（ ）*GB*。

- A. 240
- B. 250
- C. 260
- D. 270

2. 下列四个不同进制的数中，与其它三项数值上不相等的是（ ）。

- A. $(7E9)_{16}$
- B. $(2025)_{10}$
- C. $(3747)_8$
- D. $(11111101001)_2$

3. 在一个单链表中，已知 q 所指结点是 p 所指结点的前驱结点，若在 q 和 p 之间插入结点 s ，则执行（ ）。

- A. `s->next = p->next; q->next = s`
- B. `p->next = s->next; s->next = q`
- C. `q->next = s; s->next = p`
- D. `p->next = s; s->next = q`

4. 有 6 个元素 C 、 F 、 D 、 E 、 B 、 A 从左至右依次顺序入栈，在进栈过程中会有元素被弹出栈。

问下列哪一个不可能是合法的出栈序列。（ ）

- A. `C、E、D、B、F、A`
- B. `D、E、C、F、A、B`
- C. `D、F、E、B、A、C`
- D. `F、C、D、B、E、A`

5. 下列排序算法中不稳定的是哪个（ ）。

- A. 冒泡排序

B. 插入排序

C. 选择排序

D. 归并排序

6. 原字符串中任意一段连续的字符所组成的新字符串称为子串。则字符串“AAABBBCCC”共有（ ）个不同的非空子串。

A. 12

B. 24

C. 36

D. 48

7. 二叉树的前序遍历为 F,C,A,D,B,E, 中序遍历为 A,C,B,D,F,E, 则后序遍历序列是（ ）。

A. A,B,C,D,E,F

B. A,B,D,C,E,F

C. A,C,B,E,D,F

D. A,B,C,E,D,F

8. 链表相比数组的优势在于（ ）。

A. 随机访问速度快

B. 插入与删除操作效率高

C. 内存连续分配, 缓存友好

D. 不需要额外存储指针

9. 表达式 $(12 + 10) * 6 - 8 / 2$ 的后缀表达式为（ ）。

A. $12 + 10 * 6 - 8 / 2$

B. $- * + 12 10 6 / 8 2$

C. $12 10 6 8 2 + * / -$

D. $12 10 + 6 * 8 2 / -$

10. 对于下列程序, 说法正确的是（ ）。

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     string s;
5     cin>>s;
6     for(int i=0;i<s.size()-3;i++){
7         cout<<s[i];
8     }
9 }
```

A. 输入字符串为 abcde abcde, 输出为 abcde ab。

B. `s.size()` 的时间复杂度是 $O(n)$, 执行 n 次 `i<s.size()-3`, 所以整体时间为 $O(n^2)$

C. 当输入的字符串没有空格隔开时, 程序能够正确输出去掉后缀 3 个字符的字符串。

D. 如果输入的字符串长度小于 3,那么程序会产生段错误。

11. 当 $x > 0$ 时, 表达式 $(x \& (x-1)) == 0$ 用于判断 ()。

A. x 是否为偶数

B. x 是否为 2 的幂

C. x 是否为 0

D. x 的二进制表示中是否存在两个及以上的 1

12. 完全二叉树的顺序存储方案, 是指将完全二叉树的结点从上至下、从左至右依次存放到一个顺序结构的数组中。假定根结点存放在数组的 1 号位置, 则第 k 号结点的父结点如果存在的话, 应当存放在数组的 () 号位置。

A. $2k$

B. $2k + 1$

C. $k/2$ 向下取整

D. $(k + 1)/2$ 向下取整

13. 有 10 个顶点的无向图至少应该有 () 条边才能确保是一个连通图。

A. 9

B. 10

C. 11

D. 12

14. 一副纸牌除掉大小王有 52 张牌, 四种花色, 每种花色 13 张。假设从这 52 张牌中随机抽取 13 张纸牌, 则至少 () 张牌的花色一致。

A. 2

B. 3

C. 4

D. 5

15. 甲、乙、丙三位同学选修课程, 从 4 门课程中, 甲选修 2 门, 乙、丙各选修 3 门, 则不同的选修方案共有 () 种。

A. 36

B. 48

C. 96

D. 192

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题 1.5 分, 选择题 3 分, 共计 40 分。)

(1)

```
1  #include <iostream>
2  #include <cstdio>
3  using namespace std;
4  int i, j, m;
5  int a[10];
6  int main()
7  {
8      for(i = 2; i <= 6; i++)
9          a[i] = i + 1;
10     do
11     {
12         m = 2;
13         for(i = 3; i <= 6; i++)
14             if(a[m] > a[i]) m = i;
15         a[m] = a[m] + m;
16         m = 1;
17
18         for(i = 2; i <= 5; i++)
19             for(j = i + 1; j <= 6; j++)
20                 if(a[i] < a[j]) m = 0;
21     } while (m == 0);
22     printf("%d", a[2]);
23     return 0;
24 }
```

判断题

16. 程序结束时，`a[2]` 的值一定是数组 `a` 中的最大值。 ()
- A. 正确
- B. 错误
17. 第 21 行 `m==0` 成立时，数组 `a[i]` ($2 \leq i \leq 6$) 从大到小排序。 ()
- A. 正确
- B. 错误
18. 程序输出时，`a` 数组满足：对任意的 $2 \leq i < 6$ ，有 `a[i] > a[i+1]`。 ()
- A. 正确
- B. 错误
19. 删除第 16 行代码 `m=1` 程序结果会发生改变。 ()
- A. 正确
- B. 错误

选择题

20. 程序的输出结果为 ()。
- A. 58
- B. 59

C. 60

D. 61

21. 此程序的时间复杂度是 ()。

A. $O(n^3)$

B. $O(n \log n)$

C. $O(n^2)$

D. $O(n)$

(2)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int mod = 1e9 + 7;
4
5  int calc(vector<int> &a, int s) {
6      int n = a.size() - 1;
7      vector<int> dp(n+1, 0);
8      dp[0] = 1;
9      for (int i=1; i<=n; i++)
10         for (int j=1; j<=i; j++)
11             if (a[i]-a[i-j] <= s)
12                 dp[i] = (dp[i] + dp[i-j]) % mod;
13     return dp[n];
14 }
15
16 int main(){
17     int n, s;
18     cin >> n >> s;
19     vector<int> a(n+1, 0);
20     for (int i=1; i<=n; i++) {
21         cin >> a[i];
22         a[i] += a[i-1];
23     }
24     cout << calc(a, s) << endl;
25     return 0;
26 }
27
```

判断题

22. 当 $s=10$, a 数组为 $\{10, 5, 3\}$ 时输出结果为 2 ()。

A. 正确

B. 错误

23. 将第19行的 +1 删去会导致编译错误 ()。

A. 正确

B. 错误

24. 如果 s 的值小于输入的 a 数组中的最小值, 那么输出将会恒为 0 ()。

A. 正确

B. 错误

25. 若 $s=5$, a 数组为 $\{1, 2, 3, 4, 5\}$, 输出结果为 ()。

A. 1

B. 2

C. 3

D. 4

26. 若 $s=15$, a 数组为 $\{1, 2, 3, 4, 5\}$, 输出结果为 ()。

A. 4

B. 8

C. 16

D. 32

27. 若 $s=2$, a 数组所有元素均为 1, 则当 $n=15$ 时输出结果为 ()。

A. 978

B. 610

C. 987

D. 620

(3)

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  #define MOD 19260817
4  #define MAXN 1005
5  long long A[MAXN][MAXN] = {0}, sum[MAXN][MAXN] = {0};
6  int n, m, q;
7  int main() {
8      A[1][1] = A[1][0] = 1;
9      for(int i = 2; i <= 1000; i++) {
10         A[i][0] = 1;
11         for(int j = 1; j <= i; j++)
12             A[i][j] = (A[i - 1][j] + A[i - 1][j - 1]) % MOD;
13     }
14     for(int i = 1; i <= 1000; i++)
15         for(int j = 1; j <= 1000; j++)
16             sum[i][j] = (sum[i - 1][j] + sum[i][j - 1]
17                 - sum[i - 1][j - 1] + A[i][j] + MOD) % MOD;
18     int q;
19     cin >> q;
20     while(q--) {
21         int n, m;
22         cin >> n >> m;
23         cout << sum[n][m] << endl;
24     }
25     return 0;
26 }
```

判断题

28. 当 $i \leq j$ 时, $A[i][j]$ 的值是 00。 ()

A. 正确

B. 错误

29. 当 $i > j$ 时, $A[i][j]$ 的值相当于从 i 个不同元素中取出 j 个元素的排列数。 ()

A. 正确

B. 错误

30. $sum[i][j]$ 的值 ($1 < j < i \leq 1000$) 不小于 $sum[i-1][j-1]$ 的值。 ()

A. 正确

B. 错误

31. (2分)若将第 12 行改为 $A[i][j] = (A[i-1][j] + A[i-1][j-1] + MOD) \% MOD$, 程序的运行的结果不变。
()

A. 正确

B. 错误

选择题

32. (4分) $A[i][j]$ ($1 \leq i \leq 10, 1 \leq j \leq 10$) 的所有元素中, 最大值是 ()。

A. 126

B. 276

C. 252

D. 210

33. (4分) 若输入数据为 1/5 3 (其中"/"为换行符), 则输出为 ()。

A. 10

B. 35

C. 50

D. 24

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1)

(马走日) 回溯算法实际上是一个类似枚举的搜索尝试过程, 主要是在搜索过程中寻找问题的解, 当发现已不满足求解条件时, 就“回溯”返回, 尝试其他路径。

回溯法是一种选优搜索法, 按选择向前搜索以达到目标。但当搜索到某一步时, 若发现原先的选择并不优或达不到目标, 则退回一步重新选择, 这种“走不通就退回去再走”的技术称为回溯法, 而满足回溯条件的某个状态的点称为回溯点。

马在中国象棋中以日字形规则移动。

请编写一段程序, 给定 $n \times m$ 大小的棋盘以及马的初始位置 (sx, sy) , 要求不能重复走到棋盘上的同一个点, 计算马有多少途径可以遍历棋盘上的所有点。

```
1 #include <bits/stdc++.h>
```

```

2   using namespace std;
3
4   const int MAXN = 105;
5   bool vis[MAXN][MAXN];
6   int n, m, ans, t, cnt, sx, sy;
7   int dx[8] = {1, 1, 2, 2, -1, -1, -2, -2};
8   int dy[8] = {-2, 2, 1, -1, -2, 2, 1, -1};
9
10  void dfs(int x, int y){
11      if(①){
12          ans++;
13          return;
14      }
15      for(int i = 0; i < 8; i++){
16          int nx = x + dx[i], ny = y + dy[i];
17          if(② && ③){
18              vis[nx][ny] = 1;
19              cnt++;
20              dfs(④);
21              cnt--;
22              vis[nx][ny] = 0;
23          }
24      }
25  }
26  int main(){
27      cin >> t;
28      while(t--){
29          cin >> n >> m >> sx >> sy;
30          cnt = 1, ans = 0;
31          memset(vis, 0, sizeof(vis));
32          ⑤;
33          dfs(sx, sy);
34          cout << ans << "\n";
35      }
36      return 0;
37  }

```

34. ①处应填 ()。

- A. `cnt == n * m`
- B. `cnt > n * m`
- C. `ans == n * m`
- D. `cnt <= n * m`

35. ②处应填 ()。

- A. `nx >= 0 && nx < n`
- B. `nx >= 0 && nx <= n - 1 && ny >= 0 && ny <= m - 1`
- C. `nx > 0 && nx < n && ny > 0 && ny < m`
- D. `nx >= 0 || ny >= 0`

36. ③处应填 ()。

- A. `!vis[nx][ny]`

- B. `vis[nx][ny] == 1`
- C. `vis[n][m] == false`
- D. `vis[ny][nx] != 0`

37. ④ 处应填 ()。

- A. `x, y`
- B. `nx, ny`
- C. `sx, sy`
- D. `dx[i], dy[i]`

38. ⑤ 处应填 ()。

- A. `vis[sx][sy] = 1`
- B. `vis[n][m] = 1`
- C. `vis[n-1][m-1] = 1`
- D. `vis[0][0] = 1`

(2)

护卫车队在一条单行的街道前排成一队，前面河上是一座单行的桥。因为街道是一条单行道，所以任何车辆都不能超车。桥能承受一个给定的最大承载量。为了控制桥上的交通，桥两边各站一个指挥员。护卫车队被分成几个组，每组中的车辆都能同时通过该桥。当一组车队到达了桥的另一端，该端的指挥员就用电话通知另一端的指挥员，这样下一组车队才能开始通过该桥。每辆车的重量是已知的。任何一组车队

的重量之和不能超过桥的最大承重量。被分在同一组的每一辆车都以其最快的速度通过该桥。一组车队通过该桥的时间是用该车队中速度最慢的车通过该桥所需的时间来表示的。

输入格式：

输入第一行包含三个正整数（用空格隔开），第一个整数表示该桥所能承受的最大载重量 W （用吨表示），第二个整数表示该桥的长度 L （用千米表示），第三个整数表示该护卫队中车辆的总数 n （ $n < 1000$ ）

接下来的几行中，每行包含两个正整数 w_i 和 s_i （用空格隔开）， w_i 表示该车的重量（用吨表示）， s_i 表示该车过桥能达到的最快速度（用千米/小时表示）。车子的重量和速度是按车子排队等候时的顺序给出的。

问题要求计算出全部护卫车队通过该桥所需的最短时间值（用分钟表示）。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 1e3 + 5;
5  long long n, w, L, w[MAXN], s, pre[MAXN];
6  double dp[MAXN], t[MAXN][MAXN];
7  int main(){
8      cin >> w >> L >> n;
9      for(int i = 1; i <= n; i++){
10         cin >> w[i] >> s;
11         t[i][i] = ①;
12         pre[i] = ②;
13     }
14     for(int i = 1; i <= n - 1; i++){
15         for(int j = i + 1; j <= n; j++){
16             t[i][j] = ③;
17         }
18     }
19     for(int i = 1; i <= n; i++){
20         dp[i] = dp[i-1] + t[i][i];
21         for(int j = i - 1; j >= 1; j--){
22             if(④){
23                 dp[i] = min(dp[i], ⑤);
24             }else break;
25         }
26     }
27     cout << fixed << setprecision(1) << dp[n] << endl;
28     return 0;
29 }

```

39. ①处应填 ()。

- A. `L / s`
- B. `1.0 * L / s`
- C. `L / s * 60`
- D. `1.0 * L / s * 60`

40. ②处应填 ()。

- A. `pre[i+1] + w[i]`
- B. `pre[i+1] - w[i]`
- C. `pre[i-1] + w[i]`
- D. `pre[i-1] - w[i]`

41. ③处应填 ()。

- A. `max(t[i-1][j], t[i][i])`
- B. `max(t[i][j-1], t[i][i])`
- C. `max(t[i-1][j], t[j][j])`
- D. `max(t[i][j-1], t[j][j])`

42. ④处应填 ()。

- A. `pre[i] - pre[j-1] <= w`
- B. `pre[i] - pre[j] <= w`
- C. `pre[i-1] - pre[j-1] <= w`
- D. `pre[i-1] - pre[j] <= w`

43. ⑤处应填 ()。

- A. `dp[i-1] + t[i][j]`
- B. `dp[i-1] + t[j][i]`
- C. `dp[j-1] + t[i][j]`
- D. `dp[j-1] + t[j][i]`